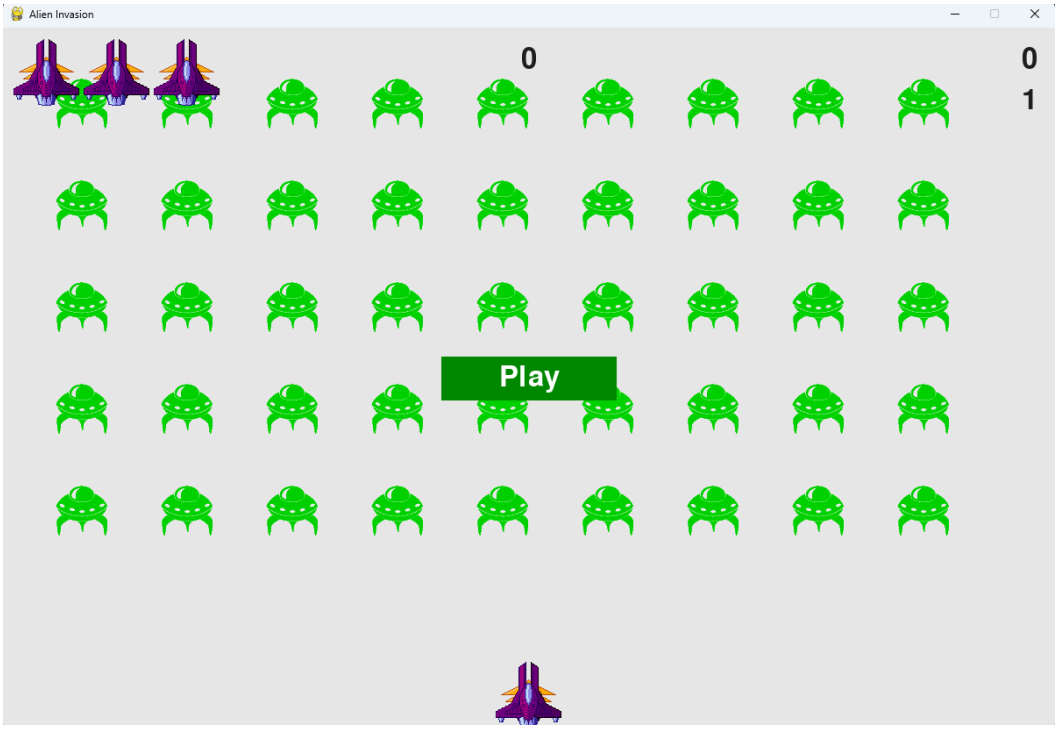
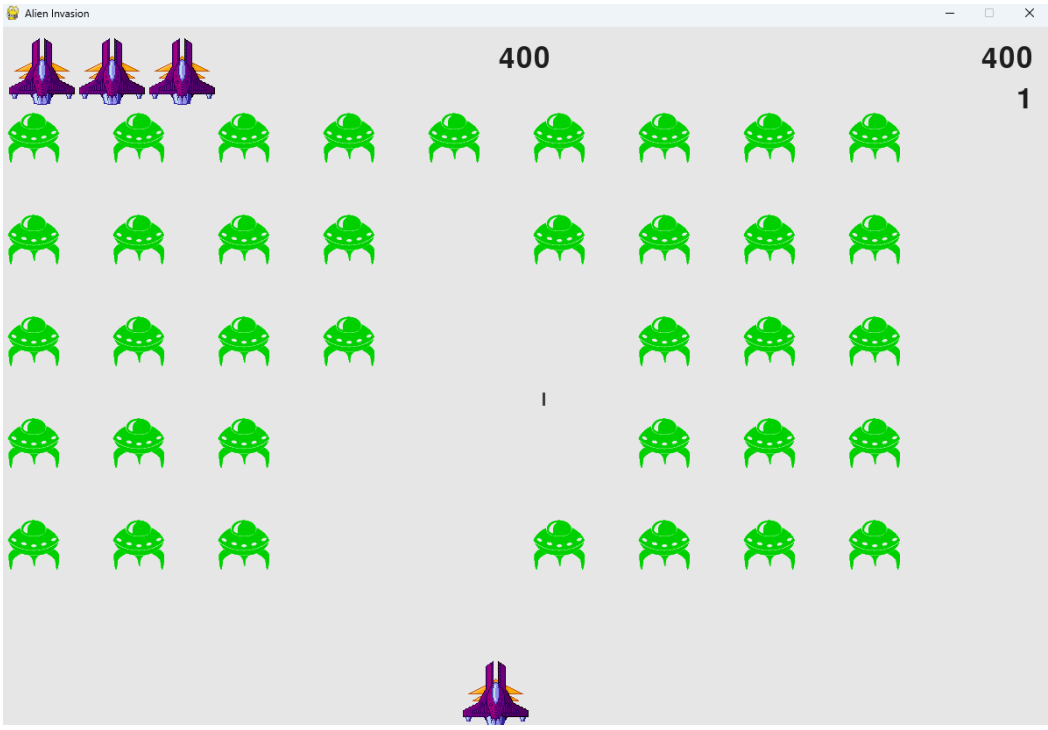
Alien Invasion

How To Play:

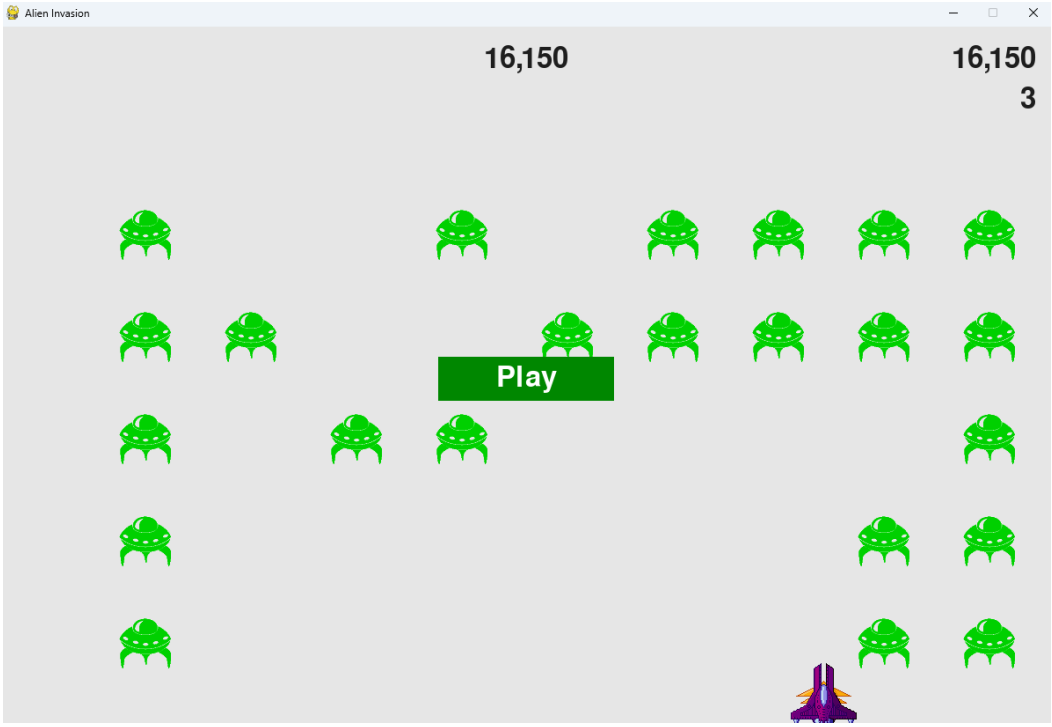
- When the Alien Invasion game starts, it will lead you to the beginning screen. To start the game, click on the Play button to start the game.



- Once the game starts, the alien ships will start to come down. Use the Spacebar to shoot bullets to destroy the alien ships and earn points. Use the left and right arrow keys to move the ship around the screen.



- When an entire fleet is destroyed, the level will increase, so will the number of points you earn from shooting down an alien and the speed of the ship and bullets.
- The game will keep on going until all three ships are gone. When the game is over, you will reset to level 1 of the game, and your high score will be shown at the top of the screen.



- To quit out of the game, you can close the window by clicking the red X in the upper right corner or by pressing 'q' on the keyboard

Resources Used:

- Python Crash Course Third Edition
- OpenGameArt.org
    - Durrr Space Ship by Tummyache
- E. H. Matthes GitHub
    - Alien.bmp

Code Printout:

alien_invasion.py

```
# All imported classes and modules needed for the game
import sys

from time import sleep

import pygame

from settings import Settings

from game_stats import GameStats

from ship import Ship

from bullet import Bullet

from alien import Alien

from button import Button
```

```
from scoreboard import Scoreboard

class AlienInvasion:
    """Overall class to manage game assets and behavior."""
    def __init__(self):
        """Initialize the game, and create game resources."""
        pygame.init()

        self.clock = pygame.time.Clock()
        self.settings = Settings()

        # Sets the size of the display screen
        self.screen = pygame.display.set_mode(
            (self.settings.screen_width,
self.settings.screen_height))
        self.settings.screen_width = self.screen.get_rect().width
        self.settings.screen_height =
self.screen.get_rect().height

        # Gives the game window the caption of 'Alien Invasion'
        pygame.display.set_caption("Alien Invasion")

        # Create an instance to store game statistics,
        #   and a scoreboard
        self.stats = GameStats(self)
        self.sb = Scoreboard(self)

        self.ship = Ship(self)
        self.bullets = pygame.sprite.Group()
        self.aliens = pygame.sprite.Group()
```

```
        self._create_fleet()

        # Start Alien Invasion in an inactive state
        self.game_active = False

        # Make the Play button
        self.play_button = Button(self, "Play")

    def run_game(self):
        """Start the main loop for the game."""
        while True:
            # Watch for keyboard and mouse events.
            self._check_events()
            if self.game_active:
                self.ship.update()
                self._update_bullets()
                self._update_aliens()

            # Keeps the screen up to date
            # and keeps it at a rate of 60 FPS
            self._update_screen()
            self.clock.tick(60)

    def _check_events(self):
        """Respond to keypresses and mouse events."""
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                sys.exit()
```

```
            elif event.type == pygame.KEYDOWN:
                self._check_keydown_events(event)
            elif event.type == pygame.KEYUP:
                self._check_keyup_events(event)
            elif event.type == pygame.MOUSEBUTTONDOWN:
                mouse_pos = pygame.mouse.get_pos()
                self._check_play_button(mouse_pos)

    def _check_play_button(self, mouse_pos):
        """Start a new game when the player clicks Play."""
        button_clicked =
self.play_button.rect.collidepoint(mouse_pos)
        if button_clicked and not self.game_active:
            # Reset the game settings
            self.settings.initialize_dynamic_settings()
            # Reset the game statistics
            self.stats.reset_stats()
            self.sb.prep_score()
            self.sb.prep_level()
            self.sb.prep_ships()
            self.game_active = True

            # Get rid of any remaining bullets and aliens.
            self.bullets.empty()
            self.aliens.empty()

            # Create a new fleet and center the ship.
            self._create_fleet()
            self.ship.center_ship()
```

```
        # Hide the mouse cursor
        pygame.mouse.set_visible(False)

    def _check_keydown_events(self, event):
        """Respond to keypresses."""
        if event.key == pygame.K_RIGHT:
            self.ship.moving_right = True
        elif event.key == pygame.K_LEFT:
            self.ship.moving_left = True
        elif event.key == pygame.K_q:
            sys.exit()
        elif event.key == pygame.K_SPACE:
            self._fire_bullet()

    def _check_keyup_events(self, event):
        """Respond to key releases"""
        if event.key == pygame.K_RIGHT:
            self.ship.moving_right = False
        elif event.key == pygame.K_LEFT:
            self.ship.moving_left = False

    def _fire_bullet(self):
        """Create a new bullet and add it to the bullets group."""
        if len(self.bullets) < self.settings.bullets_allowed:
            new_bullet = Bullet(self)
            self.bullets.add(new_bullet)

    def _create_fleet(self):
        """Create the fleet of aliens."""
```

```
        # Create an alien and keep adding alliens until there's no
room left.
        # Spacing between aliens is one alien width and one alien
height
        alien = Alien(self)
        alien_width, alien_height  = alien.rect.size

        current_x, current_y = alien_width, alien_height
        while current_y < (self.settings.screen_height - 3 *
alien_height):
            while current_x < (self.settings.screen_width - 2 *
alien_width):
                self._create_alien(current_x, current_y)
                current_x += 2 * alien_width

            # Finsihed row; reset x value, and increment y value.
            current_x = alien_width
            current_y += 2 * alien_height

    def _create_alien(self, x_position, y_position):
        """Create an alien and place it in the row."""
        new_alien  = Alien(self)
        new_alien.x = x_position
        new_alien.rect.x = x_position
        new_alien.rect.y = y_position
        self.aliens.add(new_alien)

    def _check_fleet_edges(self):
        """Respond appropriately if any aliens have reached an
edge"""
        for alien in self.aliens.sprites():
```

```
            if alien.check_edges():
                self._change_fleet_direction()
                break

    def _change_fleet_direction(self):
        """Drop the entire fleet and change the fleet's
direction"""
        for alien in self.aliens.sprites():
            alien.rect.y += self.settings.fleet_drop_speed
        self.settings.fleet_direction *= -1

    def _update_screen(self):
        """Update images on the screen, and flip to the new
screen"""
        self.screen.fill(self.settings.bg_color)
        self.ship.blitme()
        self.aliens.draw(self.screen)
        for bullet in self.bullets.sprites():
            bullet.draw_bullet()

        # Draw the score information
        self.sb.show_score()

        # Draw the play button if the game is inactive.
        if not self.game_active:
            self.play_button.draw_button()

        pygame.display.flip()

    def _update_bullets(self):
```

```
        """Update position of bullets and get rid of old
bullets"""
        # Update bullet positions
        self.bullets.update()

        # Get rid of bullets that have disappeared
        for bullet in self.bullets.copy():
            if bullet.rect.bottom <= 0:
                self.bullets.remove(bullet)

        self._check_bullet_alien_collisions()

    def _check_bullet_alien_collisions(self):
        """Respond to bullet-alien collisions."""
        # Remove any bullets and aliens that have collided
        collisions = pygame.sprite.groupcollide(
                self.bullets, self.aliens, True, True)

        # For every alien hit by a bullet per fleet
        # The points are calculated and added to the score
        if collisions:
            for aliens in collisions.values():
                self.stats.score += self.settings.alien_points *
len(aliens)
            self.sb.prep_score()
            self.sb.check_high_score()

        if not self.aliens:
            # Destroy existing bullets and create new fleet.
            self.bullets.empty()
```

```
            self._create_fleet()
            self.settings.increase_speed()

            # Increase level
            self.stats.level += 1
            self.sb.prep_level()

    def _update_aliens(self):
        """Check if the fleet is at an edge, then update
positions."""
        self._check_fleet_edges()
        self.aliens.update()

        # Look for alien-ship collisions
        if pygame.sprite.spritecollideany(self.ship, self.aliens):
            self._ship_hit()

        # Look for aliens hitting the bottom of the screen
        self._check_aliens_bottom()

    def _ship_hit(self):
        """Respond to the ship being hit by an alien."""
        if self.stats.ships_left > 0:
            # Decrement ships left, and update scoreboard
            self.stats.ships_left -= 1
            self.sb.prep_ships()

            # Get rid of any remaining bullets and aliens.
            self.bullets.empty()
            self.aliens.empty()
```

```
            # Create a new fleet and center the ship
            self._create_fleet()
            self.ship.center_ship()

            # Pause.
            sleep(0.5)
        else:
            self.game_active = False
            pygame.mouse.set_visible(True)

    def _check_aliens_bottom(self):
        """Check if any aliens have reached the bottom of the
screen."""
        for alien in self.aliens.sprites():
            if alien.rect.bottom >= self.settings.screen_height:
                # Treat this the same as if the ship got hit.
                self._ship_hit()
                break

if __name__ == '__main__':
    # Make a game instance, and run the game.
    ai = AlienInvasion()
    ai.run_game()
```

Link to my GitHub Account:

https://github.com/MZaragoza-Ramos/-Max-Zaragoza-Ramos-CSC-121-Course-Work